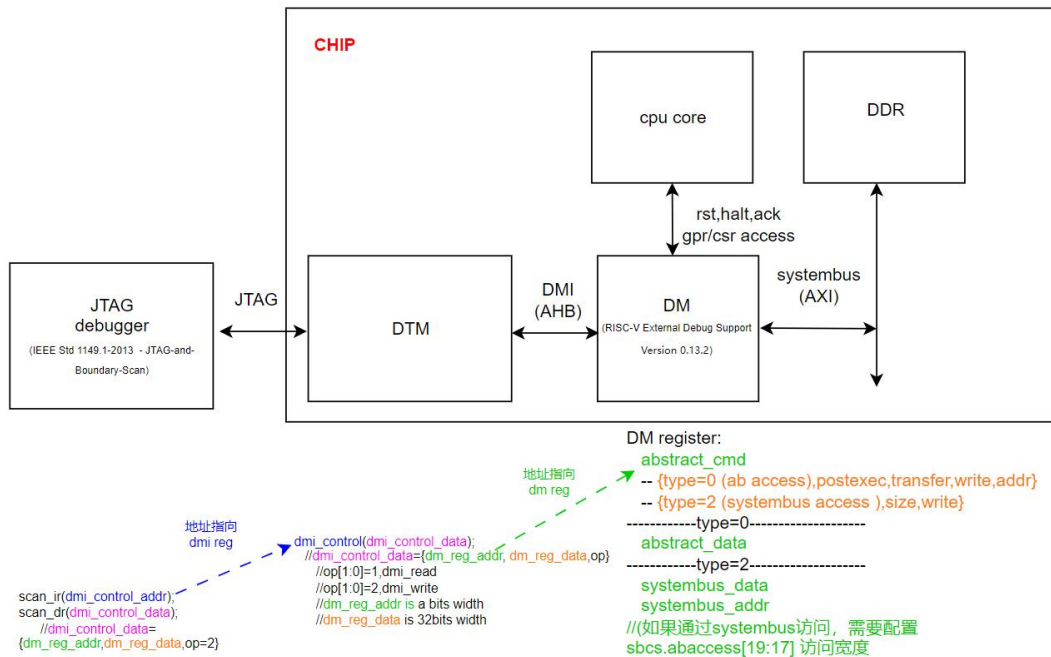


RISCV-debug

(liycstart@126.com)

1. 1. Debug & JTAG(joint Test Action Group)边界扫描



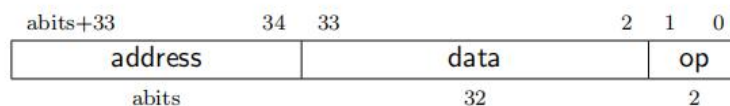
1. 1. 1. 寄存器概要

RISCV 中有 3 类 Debug 相关的寄存器:

1. **DTM 寄存器**: debugger 通过 Jtag 接口访问 DM 的寄存器 (详细查看 IEEE1149.1)。

DTM control and status (dtmcs ,at 0x10)

Debug Module Interface Access (**dmi**, at 0x11)



OP:

0: Ignore **data** and **address**. (nop)

1: Read from **address**. (read)

2: Write **data** to **address**. (write)

2. **DM 寄存器** (DM reg 地址与 DTM reg 地址不是一回事) (debugger host 通过 DTM 的 dmi

寄存器访问 DM reg ，所以 DM reg address = DTM 的 dmi.address 值)

-- DM 运行寄存器：控制 cpu hart 停止和运行

```
-> dmcontrol      at 0x10,  
-> dmstatus       at 0x11,  
-> hartinfo       at 0x12
```

-- abstract 抽象寄存器：读写 cpu hart 的寄存器 (含 GPR)

```
-> command        at 0x17,  
-> data           at 0x4,  
-> cs             at 0x16,  
-> program buf    at 0x20
```

-- systembus 寄存器：debugger 通过总线读写 mem

```
-> sbcs           at 0x38,  
-> sbaddr         at 0x39,  
-> sbdata         at 0x3c
```

3. Core debug 寄存器：hart 实现调试 (opcode 或者 abstract command 都可以访问)

```
-> dcsr           at 0x7b0  
-> dpc            at 0x7b1
```

1. 1. 2. IEEE 1149.1 标准

IEEE 1149.1 标准规定了一个四线串行接口，其端口引脚如下：

- 1.TDI (测试数据输入) (10KΩ的上拉)
- 2.TDO (测试数据输出)
- 3.TCK (测试时钟) (10KΩ的下拉电阻)
- 4.TMS (测试模式选择) (10KΩ的上拉)
- 5.TRST (测试复位) (10KΩ的上拉)，可选(若无复位引脚，则待测逻辑可由同步时钟输入复位指令而复位。

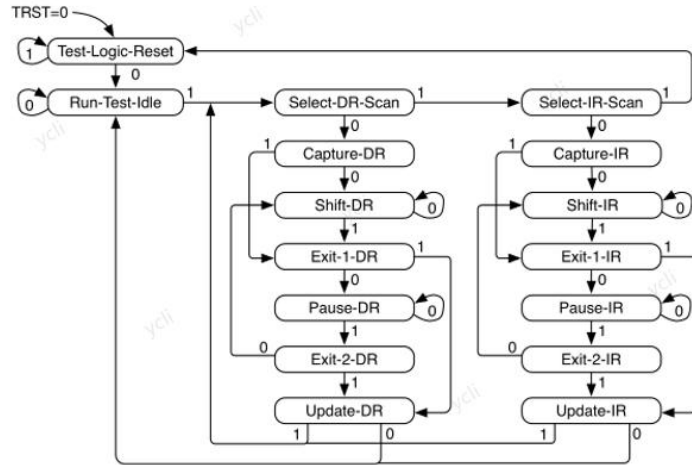
注：要想 JTAG 正常工作，所有的链上的 TAP 控制器必须处于同一状态。不管 TAP 在哪个状态，如果 TMS 在 5 个时钟周期内都保持 1，那么 TAP 都会变成 Test-Logic-Reset 状态，这便是用来同步 TAP 状态的方法。

Tck: JTAG 的时钟信号，另外三个信号 TDI、TDO、TMS 都是跟该时钟信号同步的。一般其他三根信号都是在 TCK 时钟的上升沿发生改变或者状态的切换。

TMS: 配置是通过 TMS 引脚，采用状态机的形式一次操作一位来实现的。在每个含有 JTAG 芯片内部，会有 JTAG TAP (Test Access Port 测试访问口) 控制器。下面是 TAP 控制器的状态跳转图。

常见的流程：

- 1、将指令串行输入，存到IR
- 2、将数据串行输入，存到DR
- 3、执行操作
- 4、将操作结果存到DR
- 5、将DR串行输出，TDO接收



通常使用两个最重要的状态是 **Shift-DR** 和 **Shift-IR**，两者连接 **TDI** 和 **TDO** 使用。

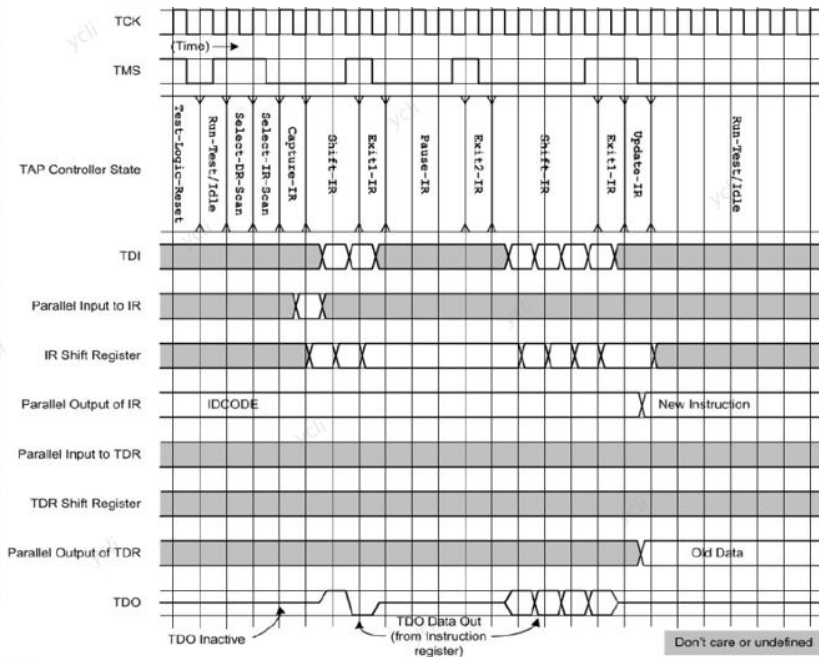
IR: Instruction register，可以写值到这个寄存器中,通知 **JTAG** 干某件事(指令)。

DR: data register

JTAG时序

更新指令

请输入正文



TAB

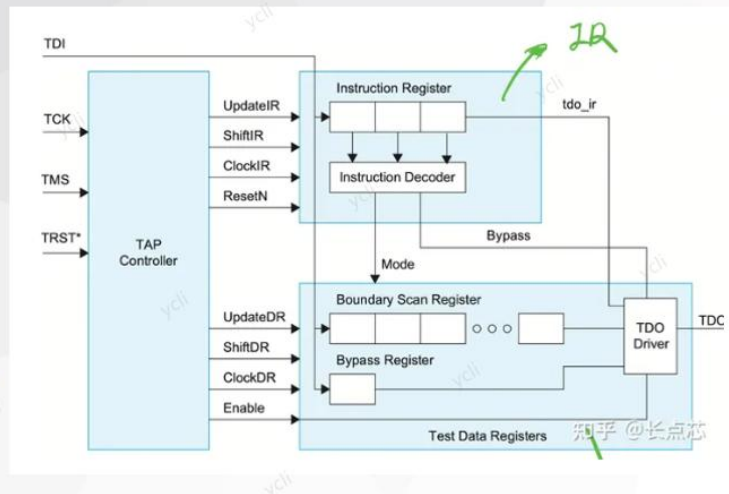
模块结构：

IR：指令寄存器

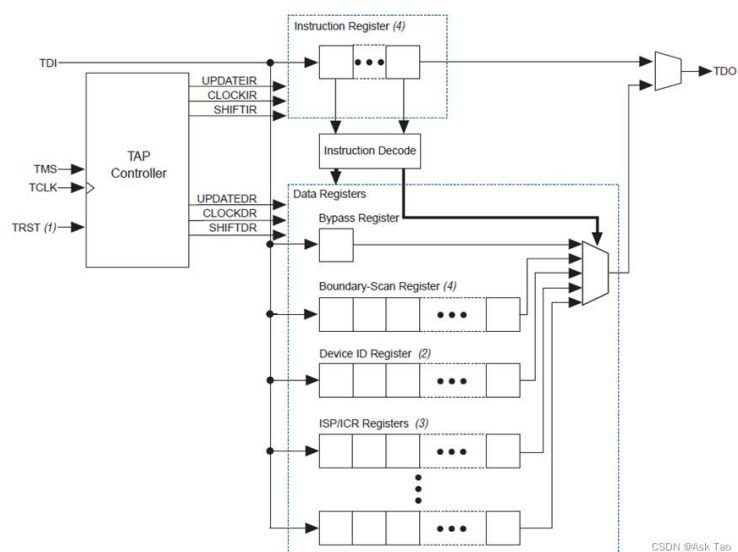
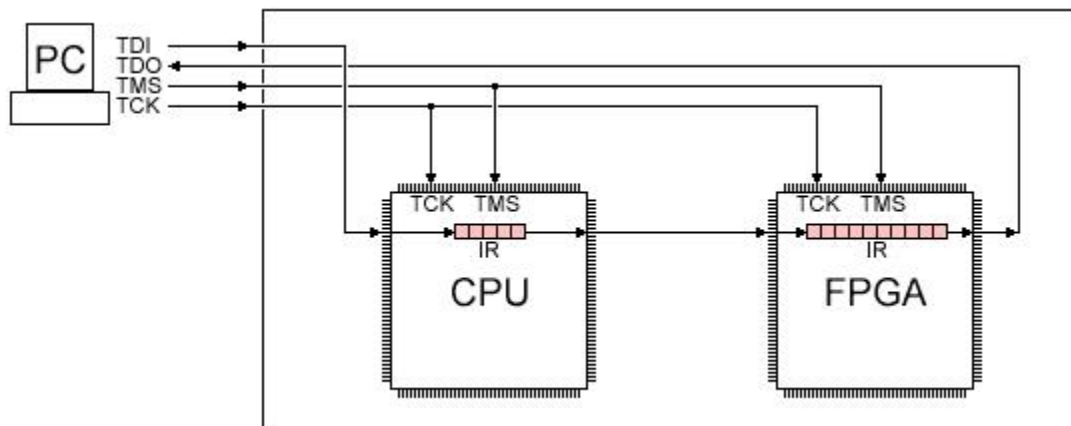
DR：数据寄存器

BSR：边界扫描寄存器

在Shift-DR状态下，在每个TCK的上升沿，TDI-移位寄存器-TDO串行通道向右移一位，TDI的数据移入移位寄存器，移位寄存器最靠近TDO的位移到TDO管脚上。



例子：



每个芯片的 TAP 控制器都只有 1 个 IR 寄存器，但是会有很多 DR 寄存器。通过读写这些寄存器，来控制芯片的每个连接引脚。

从 PC 角度看，整个链的 IR 寄存器是 15bits(5bits CPU IR 和 10bits FPGA IR)。如果想将命令写入 IR 寄存器（存入 SAMPLE 指令、EXTEST 指令等）：

- 1) PC 通过 TMS 信号线，将 TAP 控制器状态切换到 shift-IR。
- 2) 然后再通过 TDI 信号线写入 15bits 命令。

类似 IR 寄存器，写入多个 DR 寄存器。

IR=BYPASS 指令模式(IR 寄存器所有位都是 1)下，TAP 控制器对应的 DR 寄存器是个单触发器(Bypass Register 只有 1bit)，只是将 TDI 的输入数据延时 1 个时钟周期，然后通过 TDO 输出。根据这个特性，我们可以用 BYPASS 指令来计算 JTAG 链上有多少个芯片。具体地，在此指令下，每个芯片的 DR 寄存器会延时 1 个时钟周期，那么我们发送 1 个数据后，检查延时多少周期收到数据，即可知道 JTAG 链上芯片的数量。

IR=EXTEST 指令时，表示外部测试。DR 寄存器连接 I/O 单元，并且可以控制每个引脚，让输出管脚输出高低逻辑电平，或根据输入接收电平信号，检测管脚处的开路 and 短路情况。

在 RISC-V 架构中，调试功能是通过 RISC-V Debug Specification 实现的。调试器(Debugger)通过 Debug Module Interface (DMI) 与目标设备的 Debug Module (DM) 进行通信，从而访问和控制调试功能。以下是关于 DMI 配置和 Debug Bus Register 相关的流程和细节：

1. 1. 3. RISCV DM (debug module)

Debug Module (调试模块)

作用：

核心调试功能：Debug Module 是 RISC-V 调试系统的核心，负责与外部调试工具（如 JTAG 或 DMI 接口）通信，提供对处理器核心的调试控制。

调试控制：它允许调试器暂停、恢复、单步执行处理器，以及访问处理器的寄存器和内存。

调试寄存器：Debug Module 包含一组调试寄存器（如 dmstatus、dmcontrol 等），用于配置和控制调试行为。

调试中断：它还可以触发调试中断，使处理器进入调试模式。

为什么需要：

Debug Module 提供了调试的基础设施，使外部调试工具能够与处理器核心交互，实现对程序的调试和分析。

调试模块 (debug module) 是 risc-v 内核设计的比较独立也比较重要的一块，虽然 0.13.2

是 risc-v debug 的 draft 版本(<RISC-V External Debug Support Version 0.13.2.pdf>),

但目前市面上 risc-v 处理器的调试模块，几乎都是基于这个 spec 设计，在 risc-v 处理器设计中添加调试模块（电路）的目的是在设计固化后，用户仍能获取当前系统状态（通过各种资源信息）。所以调试（debug），也可以简单理解为平台各种资源信息的获取。

0.13.2 为调试协议文档，并不涉及模块的硬件实现，且真实调试硬件实现较灵活。比较主流的实现方式有两种：

- 一种是基于抽象命令（abstract command）的硬件实现。即使用抽象命令访问 gpr,csr。使用抽象命令访问 memory 或者使用 system bus 访问 memory。这种实现的优点是调试较独立，不 care 内核的执行状态。
- 另一种是基于执行的硬件实现。即只实现抽象命令访问 gpr，剩下所有的资源访问均使用执行 program buffer 中指令的方式实现。这种实现的优点是利用内核原有可以解析任何指令的特性，不破坏内核原有的数据通路。

调试的流程可以简化为：调试器发出调试访问操作，操作经 jtag bus 到达处理器的 dtm 模块，dtm 将 jtag bus 访问转换成 dmi bus 访问，dmi bus 配置 dmi 域的寄存器，dm 通过解析 dmi 寄存器完成各种调试访问。

注：在 RISC-V 架构中，reset 和 resume 是两个不同的概念，主要区别如下：

1. Reset

定义：Reset 是将系统或处理器恢复到初始状态的操作。

触发条件：通常由硬件信号（如电源启动或复位按钮）触发。

影响：

所有寄存器和状态被重置为默认值。

程序计数器（PC）跳转到预定义的复位向量地址，重新开始执行。

中断和异常状态被清除。

用途：用于系统启动或从严重错误中恢复。

2. Resume

定义：Resume 是从暂停或低功耗状态恢复到之前运行状态的操作。

触发条件：通常由软件指令或外部事件（如中断）触发。

影响：

恢复暂停前的状态，包括寄存器、程序计数器等。

从中断或暂停点继续执行。

用途：用于从低功耗模式或中断处理中恢复。

总结

Reset：完全重启，恢复到初始状态。

Resume：从暂停或低功耗状态恢复，继续之前的状态。

两者分别用于不同的场景，reset 用于彻底重启，resume 用于恢复运行。

选择 hart（哪个/哪组）

hart 的调试状态：存在(existent)/可用(avail)/运行(running)/停止(halted)

1) 运行控制（停/走）

4 种状态信息：

halt 请求 :allhalted , anyhalted;
resume ack : allresumeack ,anyresumeack;
halt on reset req: allrunning , anyrunning;
hart reset : allhavereset , and anyhavereset

控制: (debugger host)

当 Debugger 写 `dmcontrol.hartreq=1` 时，选定的 hart 被 halted。

当 Debugger 写 `dmcontrol.resumereq=1` 时，清 `dmstatus.resumeack`，向选定的 hart 发送恢复(resume)请求，然后该 hart 将 resume（收到请求后，hart 响应并 resume 恢复运行，清除 halted 信号，声明 running 信号），完成后 `resumeack=1`,`running=1`。(running harts 会忽略 `resumereq`)，无论是 halt 还是 resume 请求，hart 必须在 1 秒内响应(除非是 unavailable)。

当 Debugger 写 `dmcontrol.setresethaltreq(W1)=1` 时，hart 设置 `Hasresethaltreq=1`（复位后暂停 halt-on-reset 请求），在解复位后 hart 立即进入 debug mode。`dmstatus.hasresethaltreq` (reset-on-halt-req) 应保持为 1，直到 debugger 写 `dmcontrol.clrresethaltreq(W1)=1` 来清除 reset-on-halt-req (R)。

2) abstract

Debugger 写 (Abstract) command 前，确认 DM 写参数到 (Abstract) data0 中。

Debugger 写 (Abstract) command 前，确认 `haltreq/resumereq/ackhavereset = 0`。

Debugger 清 `abstractcs.busy=0` 前，确认 DM 写参数到 (Abstract) data0 中。

Debugger 写 `dmcontrol.hartreset/ndmreset=1` 复位 hart（但不能复位 DM 模块）

Debugger 写 `dmcontrol.dmactive=1`，复位 DM。

当 command 开始后，如果发现 hart unavailable，则 DM 会终止 command，然后设置 `busy=0`,`cmderr=4` (halt/resume)。

通过读取 `abstractcs.busy` 位来确定抽象命令是否完成。

2.1 Access register(command.cmdtype=0)

31	24	23	22	20	19	18	17	16	15	0
cmdtype	0	aarsize	aarpostincrement	postexec	transfer	write	regno			
8	1	3	1	1	1	1	16			

Debugger 通过 command 访问 CPU reg，也可通过 command 执行 Program Buffer。

当 `command.transfer =1`, `Command.write(bit16) = 0` 时，`regno` 指定的 reg -> data0（即 CPU -> DM）

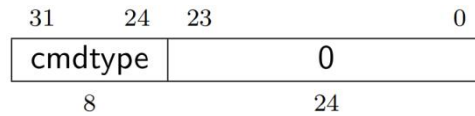
当 `command.transfer =1`, `Command.write(bit16) = 1` 时，`data0 -> regno` 指定的 reg（即 DM -> CPU）。如果访问的 reg 不存在，则 `abstractcs.cmderr=3`(exception)。

当 `Command.aarpostincrement (bit19)=1` 时，则 `Command.regno (bit15-0)` 自加。

当 Command.postexec (bit18) =1 时，则操作 program buffer。

当 Command.aarsize (访问 reg 宽度) 时，要匹配 sbcs.access(总线访问宽度)。

2.2 Quick access(command.cmdtype=1)

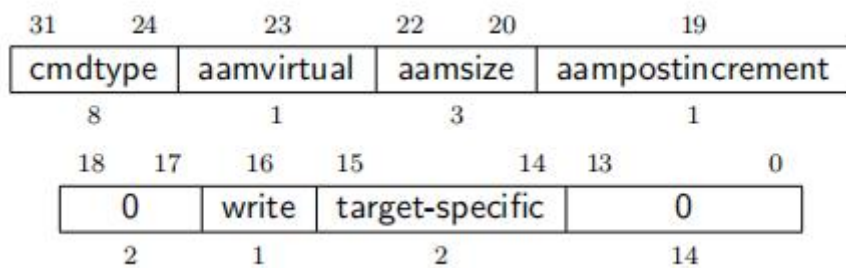


如果暂停 hart (dmcontrol.haltreq) ,且设置 abstractcs.cmderr = 4(halted/resume)。

如果执行 program buf 时，发生异常(exception),设置 abstractcs.cmderr=3(exception),则结束执行 program buf，但是 quick access 命令继续执行。(此 command 不接触 data0 reg)。

快速访问也是通过 program buffer 执行程序获得想要资源信息。但和抽象命令执行 program buffer 的区别是，快速访问，执行完 program buffer 中的指令后会自动退出调试模式，而抽象命令执行 program buffer 中的程序，执行结束后仍在调试模式中。快速访问的支持是可选实现。

2.3 Access mem(command.cmdtype=2)



如果 write=0, 则 arg1 指定的 mem ----> arg0(abstract data0)

如果 write=1, 则 arg0(abstract data0) ----> arg1 指定的 mem

如果 aampostincrement = 1,则 arg1 自加

如果以上操作 failed, 则设置 abstractcs.cmderr

注: aamvirtual=1,address 是 virtual。

也可以用作 systembus 访问 cpu 外围的寄存器。

3) Program Buffer (instruction 指令)

program buffer 用于存储一小段程序，调试器通过往 buffer 里写入一小段程序，执行，到达调试的目的。在执行程序的期间，hart 一直处于调试模式，如果有任何的指令执行异常，则剩下的指令不再执行。如果执行程序污染了 GPR 或者 CSR，在调试的最后记得恢复。

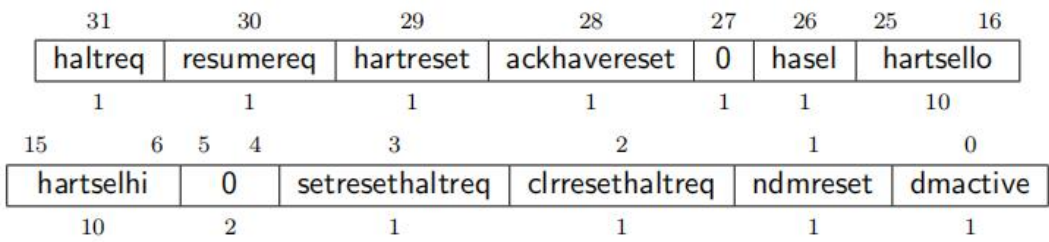
1. debugger 将 1 个小程序写入程序缓冲区(program buf)，
2. 然后使用访问寄存器抽象命令(abstract command)精确地执行它一次，并设置 command.postexec=1。
3. 程序以 ebreak 结束。

(如果 progbufsize = 1, 则 impebreak =1) program buf 能保持 32/16bits 指令，所以 debugger 只能 1 条指令的执行。

在不 halted 的情况下，一些 abstract 命令可以执行。
abstract.quick_access 可用于 halted, 执行 program buf, 再次运行 hart。

4) DM Reg

1) Debug Module Control(dmcontrol,at 0x10)



Name	Description	Access
haltreq	当前 hart 暂停请求位，置 1 会为所有当前选定的 hart 设置暂停请求位，正在运行的 hart 会在暂停请求位设置时停止。	WARZ
resumereq	当前 hart 恢复请求位，写 1 表示恢复停止的 hart。同时，它也会清除 harts 的恢复确认位，如果 haltreq 被设置，则忽略 resumereq。	W1
hartreset	写 1 表示复位当前所选 harts，写 0 表示撤销复位。（可选）	WARL
ackhavereset	写 1 表示清除当前 hart 的 havereset 状态	W1
ackunavail	写 1 表示清除当前 hart 的 unavail 状态	W1
hasel	0 表示当前只选择 1 个 hart，1 表示当前选择多个 harts	WARL

hartsello	当前选择 harts 的低 10 位， 1 位表示 1 个 hart	WARL
hartselhi	当前选择 hart 的高 10 位， 1 位表示一个 hart。 如果只有一个 hart，hartsello 的值为 1，hartselhi 的值为 0	WARL
setkeepalive	为当前选择的 hart 设置 keepalive（可选）	W1
clrkeepalive	清除当前选定的 hart 的 keepalive（可选）	W1
setresethaltreq	写 1 表示当前选择的 hart， 在复位后处于 halted 状态	W1
clrresethaltreq	写 1 表示清除 setresethaltreq	W1
ndmreset	写 1 表示复位(除 debug 部分外)所有的硬件平台， 0 表示撤销复位	R/W
dmactive	1 表示 debug 模块有效，正常调试时必须置位， 0 表示复位 debug 模块	R/W

Debug :

dret 指令，从调试模式返回。（ dpc 的值--> pc ）

dcsr.prv (U/S/M) 会指示当前的优先级(privilege)

1.1.4. Core Debug reg:

Table 4.1: Core Debug Registers

Address	Name	Page
0x7b0	Debug Control and Status (dcsr)	42
0x7b1	Debug PC (dpc)	44
0x7b2	Debug Scratch Register 0 (dscratch0)	45
0x7b3	Debug Scratch Register 1 (dscratch1)	45

可以通过 riscv 的 csr 操作码或者 DM 模块的 abstract command 访问。

dcsr (debug control and status, at 0x7b0)

prv 优先级和 cause 进入调试模式的原因，
ebreakm/s/u (0: 异常, 1: 调试模式)，
stepie(0: 中断禁止, 1: 中断启用)，
stopcount(0: 正常递增计数器, 1: 调试模式下不递增)，
stoptime(0: time 正常反映时间, 1: time 进入调试模式, 被冻结)，
mprven(0: mstatus.MPRV 被忽略, 1: enable 生效)，
step: 非调试模式下, 单步 (1 条指令) 运行。

dpc(debug PC, at 0x7b1)

进入调试模式时, 下一条地址更新到 dpc。
退出调试模式时, 硬件线程的 PC 会被更新为 dpc 中存储的虚拟地址。
进入到调试模式的 4 种原因及对应的 dpc 值:

- halt req: 进入调试模式的下一条指令地址。
- ebreak : ebreak 指令的地址
- Step : 目标地址 (pc+4)
- trigger module: time=0, 触发断点的指令的地址。
time=1, 触发断点的指令的下 1 条指令的地址。

Dscratch0(debug scratch):

可选的 scratch 寄存器可以被需要它的实现使用。
除非在 hartinfo 中有明确提到, 否则调试器禁止写入这个寄存器。

1.1.5. 触发(Trigger Module)

触发模块(TM)在 hart 内, 单独实现。

触发模块(TM)的作用是?

主要用来触发“Breakpoint exception”，并进入到 Debug Mode 或追踪（trace）程序执行的过程。

主要功能：

1. 在指定位置下 Breakpoint，当该行指令被执行时触发；
2. 在指定位置下 Watchpoint，当程序去读写那个位置时触发。

触发条件设置：

TM 用于设置硬件断点、观察点等触发条件。当处理器执行到特定指令或访问特定内存地址时，触发条件会激活，使 hart 进入调试模式。

多种触发类型：

支持多种触发类型，如指令地址匹配、数据地址匹配、数据值匹配等。

触发动作：

当触发条件满足时，Trigger Module 可以触发调试中断、暂停处理器或执行其他调试操作。

为什么需要？

TM 提供了灵活的硬件触发机制，使调试器能够在特定条件下自动触发调试行为，减少对调试器的依赖，提高调试效率。

注：为了防止未经 OS 授权的 User code 透过 Trigger，进入 debug mode，因此所有 trigger Reg 只能在 M mode 和 Debug Mode 才能被读写。

步骤：

1. Write tselect=0 (index)
2. Read tinfo, tdata.type, 判断什么时候退出 loop
3. 重复，增加 tselect 值

如果 TM“进入调试模式”和“触发断点异常”同时发生，一般情况下，是看哪个优先发生，哪个优先执行。硬件实现时，应保证外部的 debugger 和内部的执行互不影响。如果无法保证，优先“进入调试模式”，而忽略“异常”。

TM registers:

Table 5.2: Trigger Registers

Address	Name	Page
0x7a0	Trigger Select (tselect)	49
0x7a1	Trigger Data 1 (tdata1)	50
0x7a1	Match Control (mcontrol)	53
0x7a1	Instruction Count (icount)	58
0x7a1	Interrupt Trigger (itrigger)	59
0x7a1	Exception Trigger (etrigger)	60
0x7a2	Trigger Data 2 (tdata2)	50
0x7a3	Trigger Data 3 (tdata3)	51
0x7a3	Trigger Extra (RV32) (textra32)	60
0x7a3	Trigger Extra (RV64) (textra64)	61
0x7a4	Trigger Info (tinfo)	51
0x7a5	Trigger Control (tcontrol)	51
0x7a8	Machine Context (mcontext)	52
0x7aa	Supervisor Context (scontext)	52

type决定
reg field

tselect at 0x7a0: 决定访问那个触发器，可访问触发器从 index=0 开始，且连续（debug 模式和 M 模式都访问）

注意：1. 通过写后读来确认，该 index 的触发器是存在。

2. Trigger 在 M mode 也被使用，debugger 在 debug mode 下需先做好备份

tdatal at 0x7a1: 依照 type 可以分成 5 种格式：

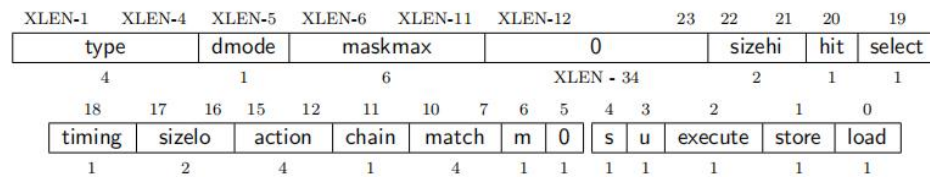
debug 模式下，XLEN=DXLEN

5.2.2 Trigger Data 1 (**tdata1**, at 0x7a1)

XLEN-1	XLEN-4	XLEN-5	XLEN-6	0
type	dmode	data		
4	1	XLEN - 5		

Field	Description	Access	Reset
type	0: There is no trigger at this tselect . 1: The trigger is a legacy SiFive address match trigger. These should not be implemented and aren't further documented here. 2: The trigger is an address/data match trigger. The remaining bits in this register act as described in mcontrol . 3: The trigger is an instruction count trigger. The remaining bits in this register act as described in icount . 4: The trigger is an interrupt trigger. The remaining bits in this register act as described in itrigger . 5: The trigger is an exception trigger. The remaining bits in this register act as described in etrigger . 15: This trigger exists (so enumeration shouldn't terminate), but is not currently available. Other values are reserved for future use.	R/W	Preset
dmode	0: Both Debug and M-mode can write the tdata registers at the selected tselect . 1: Only Debug Mode can write the tdata registers at the selected tselect . Writes from other modes are ignored. This bit is only writable from Debug Mode.	R/W	0
data	Trigger-specific data.	R/W	Preset

-- mcontrol: Match control, at 0x7a1



Type=2

dmode :

0- Debug & M mode ;

1- only Debug mode ;

Maskmax:

Select: 0-virtual addr ;

1-data value(Watch point),instruction Executed(Break point)

Time: 标示 Trigger 发生的点是在指令执行的前还是后。

Match:

0 - 当 value = tdata2 时,匹配

1 - 当 value.高 M bits = tdata2 的高 M bits 时, 匹配

2 - 当 value > tdata2 时, 匹配

3 - 当 value < tdata2 时, 匹配

4 - 当 value_lower & tdata2_upper = tdata2_lower 时, 匹配

5 - 当 value_upper & tdata2_upper = tdata2_lower 时, 匹配

Execute : breakpoint 用, 表示指令执行时启动

Store : watch point 用, 表示 data 被存储时启动

Load :watch point 用, 表示 data 被读取时启动

-- icount: Instruction Count ,at 0x7a1 (用来统计还剩多少执行的指令)

Table 5.1: **action** encoding

Value	Description
0	Raise a breakpoint exception. (Used when software wants to use the trigger module without an external debugger attached.)
1	Enter Debug Mode. (Only supported when the trigger's dmode is 1.)
2 – 5	Reserved for use by the trace specification.
other	Reserved for future use.

注：

tdatal.type=1
tdatal.**dmode**(bit Xlen-5) = 1 (only Debug mode) ,at 0x7a1

tdatal.type=2/3/4/5
mcontrol/icount/itrigger/etrigger.action ,at 0x7a1

TM 与 DM 模块的区别：

在 RISC-V 架构中，DM 和 TM 是调试系统的两个关键组成部分。
DM 负责调试的全局控制和通信，是调试系统的核心。
TM 负责具体的触发条件设置和执行，是调试系统的辅助模块。

协作关系：

DM 通过配置 TM 来设置触发条件。
当 TM 检测到触发条件满足时，会通知 DM，DM 再根据配置执行相应的调试操作（如暂停处理器）。

总结

DM 是调试系统的核心，负责与外部调试工具通信和控制处理器。
TM 是调试系统的辅助模块，负责设置和执行硬件触发条件。
两者共同协作，提供了强大的调试功能，使开发者能够高效地进行程序调试和分析。

1. 1. 6. DTM (Debug Transport Module)

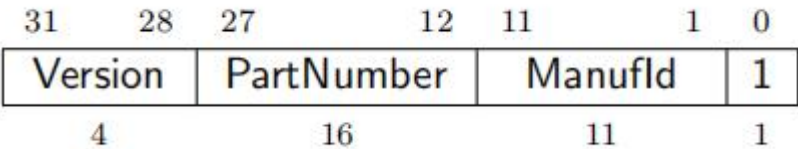
DTM 提供通过 1 个或多个传输（例如 JTAG 或 USB,可都存在，但不能同时使用）对 DM 的访

问。

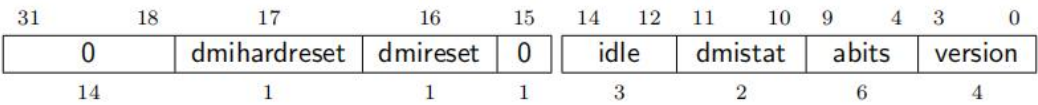
Table 6.1: JTAG DTM TAP Registers

Address	Name	Description	Page
0x00	BYPASS	JTAG recommends this encoding	64 65
0x01	IDCODE	JTAG recommends this encoding	
0x10	DTM Control and Status (dtmcs)	For Debugging	
0x11	Debug Module Interface Access (dmi)	For Debugging	
0x12	Reserved (BYPASS)	Reserved for future RISC-V debugging	
0x13	Reserved (BYPASS)	Reserved for future RISC-V debugging	
0x14	Reserved (BYPASS)	Reserved for future RISC-V debugging	
0x15	Reserved (BYPASS)	Reserved for future RISC-V standards	
0x16	Reserved (BYPASS)	Reserved for future RISC-V standards	
0x17	Reserved (BYPASS)	Reserved for future RISC-V standards	
0x1f	BYPASS	JTAG requires this encoding	

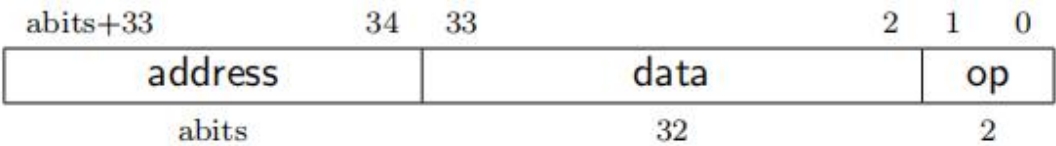
IDCODE (at 0x01)



DTM control and status (dtmcs ,at 0x10)



Debug Module Interface Access (dmi, at 0x11)



Op:

- 0: Ignore [data](#) and [address](#). (nop)
- 1: Read from [address](#). (read)
- 2: Write [data](#) to [address](#). (write)

实例:

通过 Abstract access memory comand 或 System Bus Access 命令访问 memory。
详细参考 《riscv-debug-release-0.132.pdf》 Appendix B - Debugger Implementation

Abstract command:

Read s0(GPR) using abstract command:

dmi.op	dmi.address	dmi.data	comment
Write	abstract_command (0x17)	aarize =2, transfer, regno = 0x1008	DTM Read CPU.S0 (S0 -> abstract.data0)
Read	abstract.data0	--	Returns value that was in s0

Write mstatus(CSR) using abstract command:

dmi.op	dmi.address	dmi.data	comment
Write	abstract.data0	new value	
Write	abstract_command (0x17)	aarize =2, transfer, write, regno = 0x300	write mstatus

Program buf:

Write mstatus using program buffer:

Op	Address	Value	Comment
Write	progbuf0	csrw s0, MSTATUS	
Write	progbuf1	ebreak	
Write	data0	new value	
Write	command	aarsize = 2, postexec, transfer, write, regno = 0x1008	Write s0, then execute pro- gram buffer

- 1) 先把 CPU 要执行的指令，放到 progbuf 中（包含 ebreak：在 progbuf 结束时，暂停执行）。
- 2) 要放到 S0 的值,写到 abstract_data0;
- 3) 再写 abstract_command.write=1(把 data0 的值写到 abstract_command.regno 指定的寄存器中) & abstract_command.postexec=1 (即执行 progbuf 的内容)

Read f1 using program buffer:

Op	Address	Value	Comment
Write	progbuf0	fmv.x.s s0, f1	
Write	progbuf1	ebreak	
Write	command	postexec	Execute program buffer
Write	command	transfer, regno = 0x1008	read s0
Read	data0	-	Returns the value that was in f1

- 1) 先把 CPU 要执行的指令，放到 progbuf 中（包含 ebreak：在 progbuf 结束时，暂停执行）(当执行 ebreak（指示程序缓冲区代码的结束）时，hart 返回到其停车循环)。
- 2) 再 写 abstract_command.postexec=1（即 执行 progbuf 的 内容 ）&abstract_command.write=0(把 data0 的值写到 abstract_command.regno 指定的寄存器中)
- 3) 要放到 S0 的值,写到 abstract_data0;

System bus:

Read a word from memory using system bus access:

Op	Address	Value	Comment
Write	sbc	sbaccess = 2, sbreadonaddr	Setup
Write	sbaddress0	address	
Read	sbddata0	-	Value read from memory

Read block of memory using system bus access:

Op	Address	Value	Comment
Write	sbc	sbaccess = 2, sbreadonaddr, sbreadondata, sbautoincrement	Turn on autoread and autoincrement
Write	sbaddress0	address	Writing address triggers read and increment
Read	sbddata0	-	Value read from memory
Read	sbddata0	-	Next value read from memory
...	...	...	...
Write	sbc	0	Disable autoread
Read	sbddata0	-	Get last value read from memory.

- 1) abstract command 或 Program buf ， 访问 csr/gpr
- 2) abstract command 或 Program buf 或 system bus ， 访问 mem

当 halt_req=1 时，DM 发 interrupt 到指定的 hart。Interrupt 会让 Hart 进入调试模式，并跳转到 DM 定义的 memory 区域。当出现异常（exception），PC 被保留到 dpc 中，dcsr 更新异常原因。

调试模块中的代码会导致,Hart 执行一个“park loop(停车循环)”。在 park loop 中，hart 将写 mhartid 到 DM 的 1 个存储位置,指示 hart 已经停止。为了允许 DM 单独控制几个已停止的 hart 中的一个，每个 hart 轮询存着 DM 控制（DM-controlled）的标志，以确定调试器 debugger 是否希望它执行程序缓冲区(program buffer)或恢复(resume)。

执行 abstract 命令，DM 首先根据 command，填充 program buf 的一些内部字。当 command.transfer=1 时，DM 用 lw <gpr>, 0x400(0)或 sw 0x400(0)、<gpr>填充字。64 位和 128

位访问分别使用 ld/sd 和 lq/sq。如果 command.transfer=0，则 DM 将这些指令填充为 nops。如果 command.execute=1，则继续执行调试器控制的 program buf，否则 DM 会执行 ebreak。

当执行 ebreak 时, hart 会返回 “park loop”。如果遇到异常, hart 会跳到 DM 的调试异常地址。该地址的 code 使 hart 往 DM 写 1 个 address，表示异常(该地址被认为是 fence 指令的 I/O, fence 指令，执行 Program Buffer 结果)。然后 hart 跳回 “park loop”。DM 从 hart 的写过程中，可推断出存在异常，并设置了 abstractcs.cmderr。

为了 resume，DM 设置了 1 个标志，使 hart 执行 dret 指令。当执行 dret 时，pc 将从 dpc 中恢复，并设置 dcsr.prv 的特权，恢复正常运行。

硬件的单步特性几乎与常规运行相同。调试器只是在 hart 运行之前, 设置 dcsr.step。hart 的行为与运行的情况完全相同，除了中断可能被禁用（取决于 stepie），并且它在重新进入调试模式之前，只获取和执行 1 条指令。

正常模式，访问 gpr:

通过 ld/st 指令

正常模式，访问 csr 的地址空间:

《2-riscv-privileged-20211203》的 CSR Listing 章节
Debug 模式，访问 csr 和 gpr 地址如下:

Table 3.3: Abstract Register Numbers									
0x0000 – 0x0fff			CSRs. The “PC” can be accessed here through dpc.						
0x1000 – 0x101f			GPRs						
0x1020 – 0x103f			Floating point registers						
0xc000 – 0xffff			Reserved for non-standard extensions and internal use.						

31	24	23	22	20	19	18	17	16	15	0
cmdtype		0	aarsize		aarpostincrement		postexec	transfer	write	regno
8		1	3		1		1	1	1	16

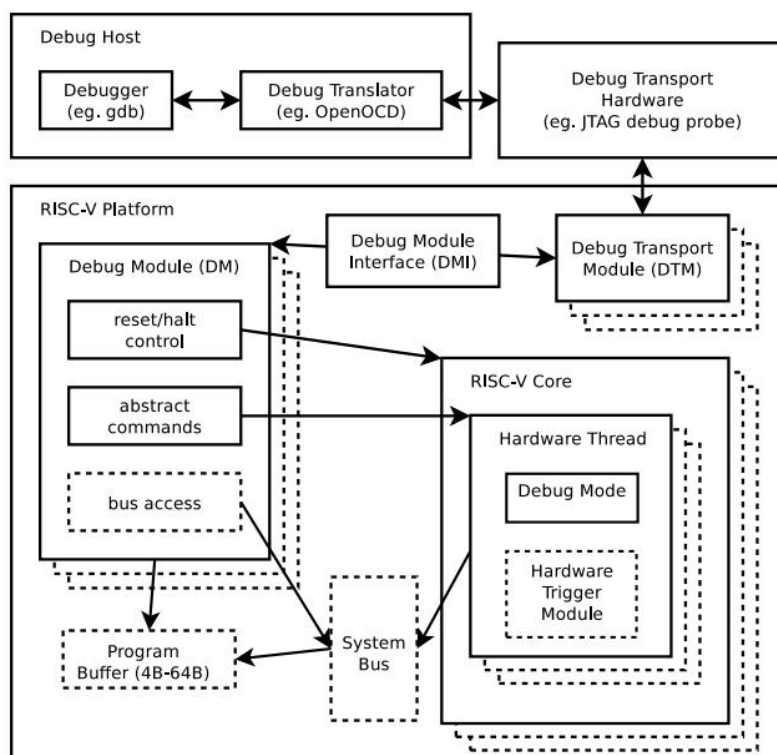
abstract command (cmdtype=0,access register)

1. RISC-V 调试架构概述

RISC-V 调试架构包括以下核心组件:

- 1) Debugger: 运行在主机上的调试软件（如 GDB）。
- 2) Debug Transport Module (DTM): 负责调试器与目标设备之间的物理通信（如 JTAG、USB）。
- 3) Debug Module (DM): 位于目标设备中，负责执行调试命令和管理调试状态。

4) Debug Bus: 用于在 DM 和其他硬件模块（如 CPU、内存）之间传输调试数据。



2. Debug Module Interface (DMI)

DMI 是调试器与 Debug Module 之间的接口, 通过 DMI 可以访问和配置 Debug Module 的寄存器。

DMI 的主要功能包括:

读写 Debug Module 的寄存器。

控制调试操作 (如启动/停止 CPU、访问内存)。

DMI 寄存器: DMI 寄存器是调试器与 Debug Module 通信的桥梁。以下是一些关键的 DMI 寄存器:

DMI Control Register: 控制 DMI 的操作模式。

DMI Data Register: 用于读写数据。

DMI Address Register: 指定要访问的 Debug Module 寄存器地址。

3. Debug Bus Register

Debug Bus 是 Debug Module 与其他硬件模块 (如 CPU、内存) 之间的数据传输通道。通过 Debug Bus, 调试器可以访问目标设备的内存、寄存器等资源。

Debug Bus 相关寄存器

以下是与 Debug Bus 相关的关键寄存器:

1. Abstract Command Register:

用于执行抽象命令 (如读写 CPU 寄存器、访问内存)。

通过 Debug Bus 将数据传输到目标硬件模块。

2. Program Buffer Register:

存储调试器发送的指令，供目标 CPU 执行。

通过 Debug Bus 将指令传输到 CPU。

3.Data Register:

用于读写目标设备的内存或寄存器。

通过 Debug Bus 传输数据。

4.Status Register:

显示 Debug Module 的当前状态（如是否忙、是否有错误）。

4. DMI 配置流程

以下是使用 DMI 配置和访问 Debug Bus Register 的典型流程：

步骤 1：初始化 DMI

通过 JTAG 或其他调试接口连接到目标设备。

配置 DMI Control Register，设置操作模式（如读写模式）。

步骤 2：访问 Debug Module 寄存器

将目标寄存器的地址写入 DMI Address Register。

通过 DMI Data Register 读写目标寄存器的值。

步骤 3：执行调试操作

通过 Abstract Command Register 发送调试命令（如读写 CPU 寄存器、访问内存）。

通过 Debug Bus 将数据传输到目标硬件模块。

步骤 4：检查状态

读取 Status Register，确认操作是否成功。

如果操作失败，检查错误信息并重新尝试。

5. 示例：通过 DMI 访问 CPU 寄存器

以下是一个通过 DMI 访问 CPU 寄存器的示例流程：

1.设置 Abstract Command Register:

写入命令：读取 CPU 寄存器。

指定目标寄存器的编号。

2.启动调试操作:

设置 DMI Control Register 的启动位。

3.等待操作完成:

轮询 Status Register，直到操作完成。

4.读取结果:

从 Data Register 中读取 CPU 寄存器的值。

6. 调试工具支持

以下工具支持 RISC-V 调试功能：

OpenOCD：开源的调试工具，支持 RISC-V 调试规范。

GDB：通过 OpenOCD 或直接与 DMI 通信，支持 RISC-V 调试。

商业调试器：如 Segger J-Link、Lauterbach TRACE32。

7. 参考文档

RISC-V Debug Specification：详细描述了 DMI、Debug Module 和 Debug Bus 的实现。

OpenOCD 文档：提供了 RISC-V 调试的配置和使用指南。

1. 1. 7. 例子:

step 调试

- 1) core_debug reg dcsr.step=1
- 2) continue(dmcontrol.resume_req=1,dmcontrol.dmactive=1)
- 3) wait(read dmstutas.allhalted=1)
- 4) core_debug reg dcsr.step=0

Quick access:(command.type=1)

Halt the hart for a minimum amount of time to perform a single memory write:

Op	Address	Value	Comment
Write	progbuf0	transfer arg2, s0	Save s0
Write	progbuf1	transfer s0, arg0	Read first argument (address)
Write	progbuf2	transfer arg0, s1	Save s1
Write	progbuf3	transfer s1, arg1	Read second argument (data)
Write	progbuf4	sw s1, 0(s0)	
Write	progbuf5	transfer s1, arg0	Restore s1
Write	progbuf6	transfer s0, arg2	Restore s0
Write	progbuf7	ebreak	
Write	data0	address	
Write	data1	data	
Write	command	0x10000000	Perform quick access

trigger:(core.debug_reg)

Enter Debug Mode just before the instruction at 0x80001234 is executed, to be used as an instruction breakpoint in ROM:

tdata1	0x105c	action=1, match=0, m=1, s=1, u=1, execute=1
tdata2	0x80001234	address

tdata1.type = 1 ,
tdata1.match=0 - 当 value = tdata2 时,匹配